SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1706**

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications.* (BTL 5)

Assessment Tool Weightage: 50%

Duration : 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Design Task

Code of Conduct:

I __V. Asha Sarthiwika(192311066)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student : V.Asha Sarthiwika

Annexure A

Design Task

1. **Hybrid AI Recommendation Engine Design**
   Design a hybrid AI-based recommendation system that integrates content-based and collaborative filtering techniques using PySpark RDDs. Explain how distributed data processing can be used to handle large-scale user-item interactions, and describe the role of intelligent agents in adapting recommendations based on user behavior and feedback in real time.

2. **Smart Disaster Detection System**
   Develop a distributed AI system using PySpark and RDD operations to detect natural disasters (such as floods or earthquakes) from multi-source data streams (satellite images, sensors, and social media). Describe the system architecture, data fusion techniques, and how intelligent agents coordinate to provide early warnings and response strategies.

3. **AI-Based Fraud Detection Framework**
   Design a scalable fraud detection system using PySpark RDDs to process large volumes of financial transactions. Explain how anomaly detection algorithms can be implemented in a distributed manner and how intelligent agents collaborate to identify suspicious patterns, trigger alerts, and continuously improve detection accuracy.

4. **Autonomous Supply Chain Optimization System**
   Construct a distributed AI system using PySpark for optimizing supply chain operations (inventory, logistics, demand forecasting). Describe how RDD transformations support large-scale data processing and how multiple intelligent agents interact to make decentralized decisions for cost minimization and efficiency improvement.

5. **Personalized Learning Analytics Platform**
   Design an AI-driven educational platform that uses PySpark and RDDs to analyze student learning behavior and performance data. Explain how the system supports adaptive learning through intelligent agents, real-time feedback, and scalable analytics to personalize content delivery for thousands of learners.

RUBRICS FOR CALCULATION

Evaluation Criteria	Problem Understanding & Requirement Analysis	System Architecture Design	Use of PySpark & RDD Operations	Intelligent Agent Integration	Scalability & Distributed Computing Design	Data Flow & Processing Pipeline	Innovation & Creativity	Clarity, Presentation & Documentation
Weightage	10%	20%	15%	15%	10%	10%	10%	10%

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)
Problem Understanding & Requirement Analysis	Clearly defines problem, objectives, and constraints	Mostly clear with minor gaps	Basic understanding	Unclear or incorrect
System Architecture Design	Complete, scalable, well-structured architecture with diagrams	Good design with minor issues	Basic design, lacks detail	Poor/incomplete
Use of PySpark & RDD Operations	Efficient and correct use of RDD transformations/actions	Mostly correct usage	Limited or partial use	Incorrect/missing
Intelligent Agent Integration	Strong integration with clear agent roles and logic	Good integration	Basic integration	Poor/no integration
Scalability & Distributed Computing Design	Clearly addresses scalability, fault tolerance, parallelism	Covers most aspects	Limited consideration	Not addressed

Data Flow & Processing Pipeline	Clear, logical, and well-defined data flow	Mostly clear	Some gaps	Poorly defined
Innovation & Creativity	Highly innovative, realistic, and impactful solution	Some innovation	Basic idea	No originality
Clarity, Presentation & Documentation	Very clear, neat diagrams, well-organized report	Mostly clear	Somewhat organized	Poor presentation

ANSWERS:

Q1. Hybrid AI Recommendation Engine Design

1. Problem

A recommendation system must suggest relevant products, movies, courses, or services to users. It should combine content-based filtering and collaborative filtering and process large-scale user-item data using PySpark RDDs.

2. Objectives

- Recommend personalized items to users.
- Combine content information and user behavior.
- Process millions of user-item interactions efficiently.
- Adapt recommendations based on feedback**.**

3. System Architecture

Data Sources → PySpark RDD → Data Processing → Recommendation Models → Intelligent Agent → Recommendations

4. Content-Based Filtering

Content-based filtering recommends items similar to those previously liked by the user. Item features such as category, keywords, rating, or description are used**.**

5. Collaborative Filtering

Collaborative filtering identifies users with similar preferences and recommends items liked by similar users.

6. PySpark RDD Operations

RDDs can store large-scale user-item interaction data.

Important operations:

- map() – convert data into key-value pairs.
- filter() – remove invalid records.
- groupByKey() – group interactions.
- reduceByKey() – combine ratings or preferences.
- join() – combine user and item information.

7. Intelligent Agent

The recommendation agent monitors user behavior such as clicks, ratings, searches, and purchases. It updates recommendations when new feedback is received.

8. Scalability

PySpark distributes RDD processing across multiple worker nodes. Therefore, millions of interactions can be processed in parallel.

9. Expected Output

The system provides a ranked list of personalized recommendations for every user.

10. Conclusion

A hybrid recommendation engine combines the advantages of content-based and collaborative filtering. PySpark provides scalable distributed processing, while intelligent agents continuously adapt recommendations according to user behavior.

Q2. Smart Disaster Detection System

1. Problem

Natural disasters such as floods and earthquakes need to be detected quickly using information from satellite images, sensors, and social media.

2. Objectives

- Collect disaster-related data from multiple sources.
- Detect abnormal or dangerous conditions.
- Combine information from different sources.
- Provide early warnings.

3. System Architecture

Satellite + Sensors + Social Media → PySpark RDD → Data Cleaning → Data Fusion → Disaster Detection → Intelligent Agents → Alert

4. Data Collection

Satellite images provide geographical information, sensors provide environmental measurements, and social media provides real-time public reports.

5. Data Preprocessing

PySpark RDDs can remove invalid data, filter unnecessary information, and transform the collected data into a common format.

6. Data Fusion

Data from different sources is combined to improve detection accuracy. For example, sensor readings can be compared with satellite observations and social media reports.

7. PySpark RDD Operations

- map() – transform incoming records.
- filter() – identify abnormal values.
- reduceByKey() – aggregate regional information.
- groupByKey() – group disaster reports by location.

8. Intelligent Agents

Different agents can monitor sensors, satellite information, and public reports. A decision agent combines their information and determines whether an emergency warning is required.

9. Scalability

PySpark distributes large amounts of sensor, satellite, and social-media data across multiple machines for parallel processing.

10. Conclusion

The proposed system provides fast disaster detection by combining multi-source data, distributed PySpark processing, and intelligent agents for early warning and response strategies.

Q3. AI-Based Fraud Detection Framework

1. Problem

Financial institutions process huge numbers of transactions. The system must identify suspicious transactions efficiently and detect unusual patterns.

2. Objectives

- Process large volumes of financial transactions.
- Detect abnormal transaction behavior.

- Identify suspicious patterns.
- Generate alerts automatically.

3. System Architecture

Transaction Data → PySpark RDD → Preprocessing → Feature Extraction → Anomaly Detection → Intelligent Agents → Alert

4. Data Processing

Transaction details such as amount, location, time, frequency, and user behavior are processed to identify unusual activities.

5. Anomaly Detection

Transactions that significantly differ from normal user behavior are treated as potential fraudulent activities.

6. PySpark RDD Operations

- map() – extract transaction features.
- filter() – remove invalid transactions.
- groupByKey() – group transactions by customer.
- reduceByKey() – calculate transaction statistics.
- join() – combine transaction and customer information.

7. Distributed Processing

RDDs divide the transaction data across multiple nodes. Each node processes a portion of the data simultaneously.

8. Intelligent Agents

A monitoring agent observes transactions, a detection agent identifies suspicious patterns, and an alert agent informs the appropriate authority.

9. Continuous Improvement

Feedback from confirmed fraudulent and genuine transactions can be used to improve future detection accuracy.

10. Conclusion

A PySpark-based fraud detection system can process large transaction datasets efficiently. Distributed anomaly detection combined with intelligent agents enables fast identification of suspicious activities and automatic alerts.

Q4. Autonomous Supply Chain Optimization System

1. Problem

Supply chains involve inventory, transportation, logistics, and demand forecasting. A distributed AI system can optimize these operations and reduce cost**.**

2. Objectives

- Maintain appropriate inventory levels.
- Optimize transportation and logistics.
- Forecast product demand.
- Minimize cost and improve efficiency**.**

3. System Architecture

Sales + Inventory + Transport Data → PySpark RDD → Processing → Demand Forecasting → Optimization → Intelligent Agents → Decisions

4. Data Collection

The system collects sales records, inventory levels, delivery information, supplier data, and customer demand.

5. PySpark RDD Processing

RDDs process large-scale supply-chain records.

Operations include:

- map() – transform supply-chain data.
- filter() – remove invalid records.
- groupByKey() – group data by product or location.
- reduceByKey() – calculate total demand or inventory.

6. Demand Forecasting

Historical sales data is analyzed to estimate future demand. This helps prevent overstocking and shortages.

7. Intelligent Agents

Different agents can manage inventory, transportation, suppliers, and demand forecasting. They communicate with each other to make decentralized decisions.

8. Cost Optimization

Agents select suitable inventory and transportation decisions to reduce storage, transportation, and operational costs.

9. Scalability

PySpark distributes large supply-chain datasets across multiple nodes, allowing parallel processing and faster decision-making**.**

10. Conclusion

An autonomous supply-chain system combines PySpark, RDD operations, forecasting, and intelligent agents to improve logistics, minimize costs, and increase overall supply-chain efficiency**.**

Q5. Personalized Learning Analytics Platform

1. Problem

An educational platform needs to analyze student learning behavior and performance and provide personalized content to thousands of learners.

2. Objectives

- Analyze student learning behavior.
- Identify strengths and weaknesses.
- Provide personalized learning materials.
- Give real-time feedback.

3. System Architecture

Student Data → PySpark RDD → Data Processing → Performance Analysis → Intelligent Agent → Personalized Content → Feedback

4. Data Collection

The system collects quiz scores, assignment performance, attendance, learning time, course progress, and interaction data**.**

5. PySpark RDD Processing

RDDs are used to process large amounts of student data.

Operations include:

- map() – transform student records.
- filter() – select required records.

- groupByKey() – group data by student or course.
- reduceByKey() – calculate performance statistics**.**

6. Learning Analysis

The system identifies topics where students perform well or require additional practice**.**

7. Intelligent Agents

A learning agent observes student performance and recommends suitable learning materials. It can adjust the difficulty according to the learner's progress**.**

8. Real-Time Feedback

When a student completes a quiz or activity, the agent analyzes the result and provides suitable feedback or recommends additional content**.**

9. Scalability

PySpark allows student data from thousands of learners to be processed in parallel across distributed systems.

10. Conclusion

The personalized learning platform combines scalable PySpark analytics with intelligent agents to provide adaptive content, real-time feedback, and personalized learning experiences for large numbers of students.